Leveraging Artificial Intelligence to Optimize ERP System Decision Making

HTL Spengergasse
Mojmír Horváth
horvathmojmir@gmail.com

April 5, 2026

Contents

1 Abstract **2**
1.1 Deutsch . . . 2
1.2 English . . . 2

2 Background and Related Work **2**
2.1 ERP Systems Overview . . . 2
2.2 Data Integration Challenges . . . 3
2.3 AI in ERP Applications . . . 3

3 Theoretical Foundations **3**
3.1 What is AI? . . . 3
3.2 Relevant Techniques for Data Harmonization . . . 3
3.3 Rule-Based Systems vs. Machine Learning . . . 4
3.4 Types of Demand Forecasting . . . 4
3.4.1 Time Series Analysis . . . 4
3.4.2 Causal Models . . . 5
3.4.3 Machine Learning Approaches . . . 5

4 System AI Connector Architecture **5**
4.1 Functionality Overview . . . 5
4.2 Pipeline Architecture and Data Flow . . . 5
4.3 Layer 1: Data Normalization Layer (Adapter) . . . 7
4.4 Layer 2: Rule Engine . . . 7
4.5 Layer 3: ML Engine . . . 7
4.6 Layer 4: Dependency Synthesis . . . 8
4.7 Layer 5: Validation . . . 8
4.8 Layer 6: LLM Fallback Engine . . . 8
4.9 Layer 7: Backend Integration . . . 9

5 Design Justifications and Trade-offs **9**
5.1 When Does Type-Agnostic ERP Parsing Work? . . . 9
5.2 Impact of Fuzzy Matching on Accuracy . . . 10
5.3 Local LLM Performance vs. Cloud API . . . 10
5.4 Choice of Small Local LLMs . . . 12

6 Evaluation **13**
6.1 Test Datasets . . . 13
6.2 Illustrative Example: Parsing a Foreign ERP Record . . . 13
6.3 Metrics . . . 16
6.4 Results . . . 16
6.5 Evaluation of Decision Quality . . . 17

7 Discussion **17**
7.1 Hybrid vs. Pure Rule-Based Performance . . . 17
7.2 Latency and Adaptability Trade-offs . . . 18
7.3 System Limitations . . . 19
7.4 Future Work . . . 19

8 Conclusion **20**

1 Abstract

1.1 Deutsch

In der Praxis weisen ERP-Systeme (Enterprise Resource Planning) stark heterogene Datenschemata zwischen verschiedenen Anbietern (SAP S/4HANA, Odoo, Oracle NetSuite, Microsoft Dynamics 365, Epicor, Infor) auf. Dies macht die systemübergreifende Integration fehleranfällig, teuer und langfristig schwer wartbar. Diese Arbeit präsentiert den Prototyp eines KI-gestützten Konnektor-Dienstes, der die Datenaufnahme, Normalisierung und das Routing von ERP-Daten automatisiert. Dabei kommt eine dreistufige, hybride Pipeline zum Einsatz: deterministisches, regelbasiertes Mapping, auf maschinellem Lernen basierendes Fuzzy-Matching von Werten sowie ein lokales Large Language Model (LLM) als Fallback-Ebene. Der Konnektor wurde in Python (FastAPI, Pydantic, RapidFuzz, scikit-learn) implementiert, nutzt ein lokal gehostetes LLM (Ollama/Qwen2.5-Coder) und ist an ein auf Spring Boot basierendes ERP-Backend angebunden. Die Auswertung anhand synthetischer ERP-Datensätze, die Schemata mehrerer namhafter Anbieter umfassen, zeigt, dass die hybride Pipeline im Vergleich zu rein regelbasierten Ansätzen hohe Erfolgsraten bei der Datentransformation erzielt und sich besser an unregelmäßige oder bisher unbekannte Schemata anpasst. Die Ergebnisse verdeutlichen, dass das KI-gestützte Parsen fremder ERP-Exporte in ein einheitliches DTO-Schema den Integrationsaufwand erheblich reduzieren kann, während gleichzeitig die Erweiterbarkeit für zukünftige Funktionen wie Nachfrageprognosen und Anomalieerkennung erhalten bleibt.

1.2 English

Enterprise Resource Planning (ERP) systems in real-world deployments suffer from heterogeneous data schemas across vendors (SAP S/4HANA, Odoo, Oracle NetSuite, Microsoft Dynamics 365, Epicor, Infor), which makes cross-system integration brittle, expensive, and difficult to maintain over time. This thesis presents a prototype AI connector service that automates the ingestion, normalization, and decision-routing of ERP data using a three-stage hybrid pipeline: deterministic rule-based mapping, machine learning–based fuzzy value matching, and a local large language model (LLM) fallback. The connector is implemented in Python using FastAPI, Pydantic, RapidFuzz, and scikit-learn, combined with a locally hosted LLM (Ollama/Qwen2.5-Coder), and is integrated with a Spring Boot–based ERP backend. Evaluation on synthetic ERP datasets comprising multiple major vendor schemas shows that the hybrid pipeline achieves high transformation success rates and improved adaptability to irregular or previously unseen schemas compared to purely rule-based approaches. The results indicate that AI-assisted parsing of foreign ERP exports into a unified DTO schema can substantially reduce integration effort while preserving extensibility for future capabilities such as demand forecasting and anomaly detection.

2 Background and Related Work

2.1 ERP Systems Overview

Enterprise Resource Planning (ERP) systems provide integrated support for core business functions such as finance, supply chain, manufacturing, and human resources, typically through a modular architecture connected to a central data model.[31] Major vendors, including SAP S/4HANA, Odoo, Oracle NetSuite, Microsoft Dynamics 365, Epicor Kinetic, and Infor CloudSuite, offer overlapping functional coverage but expose distinct data schemas, APIs, and customization mechanisms, which complicates cross-system interoperability.[10, 22]

2.2 Data Integration Challenges

Integrating data across heterogeneous ERP ecosystems requires reconciling differences in schema design, naming conventions, and modeling choices.[11] Common obstacles include non-standardized field names, varying levels of normalization and nesting (for example, deeply nested JSON vs. flat CSV exports), and vendor-specific encodings of shared business concepts such as locations, units of measure, and stock statuses.[28, 30] These factors increase the cost of point-to-point integrations and motivate reusable harmonization layers like the AI connector developed in this thesis.

2.3 AI in ERP Applications

In recent years, AI techniques have been applied to ERP data for tasks such as demand forecasting, anomaly detection in transactional logs, intelligent routing of workflows, and recommendation of corrective actions.[31, 18] Contemporary work emphasizes embedding machine learning and, increasingly, large language models into enterprise data pipelines so that systems can learn from historical patterns while still respecting existing process controls and audit requirements.[4, 32] The connector presented here positions AI not only as an analytical layer but also as an operational integration component that automates the ingestion and normalization of multi-vendor ERP data.

3 Theoretical Foundations

3.1 What is AI?

Artificial Intelligence (AI) refers to computational systems capable of performing tasks that require human-like cognitive functions, such as learning, reasoning, and adaptation [27]. Within enterprise software, AI encompasses the ability to extract meaning from business data, infer relationships between heterogeneous schemas, and support decision-making through predictive or generative methods. In modern ERP systems, AI serves as a layer of abstraction that can reconcile structured and semi-structured data, automate repetitive transformations, and enhance adaptability in dynamic enterprise environments [31]. This research applies AI as an integrative mechanism to align inconsistent data structures between ERP systems, enabling rule-to-learning transitions when deterministic approaches fail.

3.2 Relevant Techniques for Data Harmonization

Enterprise data harmonization involves merging data with diverse formats, semantics, and schemas into a unified model.[11] At a high level, harmonization can be viewed as learning or encoding a mapping function $f : \mathcal{X} \to \mathcal{Y}$ from heterogeneous source representations $\mathcal{X}$ (e.g., vendor-specific ERP exports) to a recognized target schema $\mathcal{Y}$, subject to constraints such as referential integrity and type correctness. Multiple paradigms underpin this process:

- **Rule engines**: Deterministic mapping systems encode transformation knowledge as static rules, typically of the form "IF condition THEN action" over input records.[12, 14] In the ERP harmonization setting, a rule might map a vendor key `"MaterialNumber"` to the recognized `"sku"` field if it appears in a product context and satisfies simple type checks. Because all logic is explicitly specified, rule engines provide full interpretability and reproducibility: given identical input, the mapping function f always produces the same output. However, this also implies a "knowledge acquisition bottleneck", each new schema variant requires manual authoring and maintenance of additional rules, and the rule base can grow combinatorially with the number of vendors and edge cases.

- **Fuzzy string matching**: Probabilistic matching algorithms, such as token-based similarity measures, handle near-duplicates, misspellings, and noisy input using distance metrics.[5] Formally, they define a similarity function $s : \Sigma^* \times \Sigma^* \to [0, 1]$ over strings, where values close to 1 indicate strong correspondence (for example, a high token sort ratio between "Warehouse A - North" and "North Warehouse A"). In this prototype, a library such as RapidFuzz computes token-based scores and selects the best recognized label for a given observed value when s exceeds a threshold.[20, 25] These methods are lightweight yet effective for aligning ERP field labels and categorical values (units of measure, location names, country codes) without enumerating every spelling or formatting variant in the rule base.
- **Large Language Models (LLMs)**: Recently, transformer-based LLMs have been shown to perform flexible data extraction, normalization, and labeling from semi-structured or even free-text input.[4, 32] Conceptually, an LLM implements a conditional sequence-to-sequence model $g_\theta(x) \approx y$, where x is raw or loosely structured text and y is a structured representation such as JSON that conforms to a target schema. Within ERP data pipelines, LLMs offer redundancy for failed deterministic or statistical mappings by leveraging contextual understanding: for example, inferring that a field labeled "WH" in the vicinity of inventory quantities likely denotes a warehouse code. At the same time, LLMs can hallucinate fields or over-generalize patterns, so in this connector they are positioned as a controlled fallback layer rather than the primary mechanism for schema alignment.

The hybrid pipeline implemented in this prototype therefore mirrors an evolution from symbolic AI (deterministic rules) toward connectionist systems (ML and neural models), with each paradigm addressing a different part of the harmonization problem.[15, 16] Rule engines capture stable, high-confidence mappings that must remain transparent to integrators, fuzzy matching absorbs the long tail of noisy but structurally simple variations, and LLMs provide a flexible but carefully sandboxed mechanism to recover from residual ambiguities that escape both deterministic and classical statistical techniques.

3.3 Rule-Based Systems vs. Machine Learning

Rule-based systems operate through explicitly codified logic, ensuring reproducibility and interpretability [14]. However, as the diversity of data sources grows, purely rule-based architectures exhibit exponential growth in maintenance overhead. They fail to generalize to unseen schemas or contextual variations. Machine learning (ML), by contrast, infers transformation patterns from examples rather than from hand-written logic [15]. In ERP data integration, ML enables schema alignment and entity-linking even when semantic or lexical mismatches occur. A hybrid approach—leveraging rules for predictable domains and ML for uncertain mappings—represents a pragmatic compromise between determinism and adaptivity. This mixed architecture is consistent with current AI trends in enterprise systems that emphasize interpretable automation, model transparency, and feedback-driven learning loops [16].

3.4 Types of Demand Forecasting

Demand forecasting is a critical element in ERP-driven decision optimization. It aims to predict future product demand based on historical and contextual factors.

3.4.1 Time Series Analysis

Classical time series modeling techniques, such as ARIMA and exponential smoothing, model temporal dependencies and seasonality [18]. These methods are statistically efficient and well-

suited for stationary patterns in inventory and sales data.

3.4.2 Causal Models

Causal or econometric models incorporate exogenous predictors (e.g., marketing activity, macroeconomic factors) to estimate how external conditions affect demand [29]. Such models suit enterprise planning tasks requiring interpretability and policy sensitivity.

3.4.3 Machine Learning Approaches

Recent advances integrate ML and deep learning architectures into demand forecasting, improving nonlinear pattern recognition and feature interactions [3]. Recurrent (LSTM/GRU) or transformer-based models capture long-term dependencies, outperforming traditional methods on complex datasets. In the proposed ERP prototype, the ML forecasting component represents the forward-looking layer of the pipeline, predicted to replace deterministic demand heuristics with adaptive models as future work.

4 System AI Connector Architecture

4.1 Functionality Overview

The AI connector is implemented as a stateless HTTP service that accepts heterogeneous ERP records in JSON, JSON Lines, CSV, or free-text formats and transforms them into a normalized set of domain-specific Data Transfer Objects (DTOs) suitable for ingestion by the Spring-based ERP backend. Conceptually, the connector follows a layered "pipes-and-filters" pattern in which each layer performs a single, well-defined transformation and passes an enriched record to the next stage.[17, 22] This design decouples concerns such as parsing, schema alignment, semantic normalization, validation, and persistence, enabling independent testing and targeted optimization of each layer.

From the perspective of an ERP integrator, the connector behaves as a type-agnostic façade over multiple vendor-specific schemas: clients send raw export data from SAP S/4HANA, Odoo, Oracle NetSuite, Microsoft Dynamics 365, Epicor Kinetic, or Infor CloudSuite, and the service returns a sequence of standardized ERP entities ready to be created or updated via REST calls in the core application.[10] This reduces the need for bespoke integration code in the ERP itself and centralizes data harmonization logic in a single, reusable component.

4.2 Pipeline Architecture and Data Flow

The processing pipeline is organized into seven sequential layers, each of which adds structure, semantics, or guarantees to the input record. The execution flow is illustrated in Figure 1.

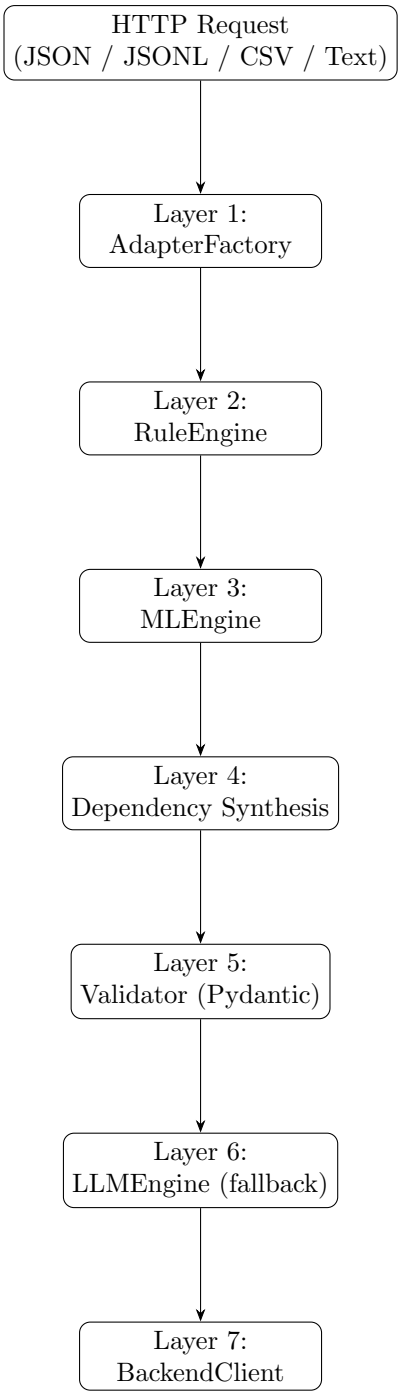

Figure 1: Layered architecture of the AI connector pipeline.

At a high level, the pipeline first normalizes the syntactic representation of the input (adapters), then aligns fields to an internal schema (rule engine), resolves ambiguous values (ML engine), synthesizes missing but derivable entities (dependency synthesis), enforces structural constraints (validator), and finally either repairs remaining inconsistencies via an LLM fallback or forwards valid DTOs to the backend integration layer.[9] The layered decomposition mirrors modern data pipeline architectures used in large-scale enterprise data integration, with explicit attention to idempotency, observability, and failure isolation.[22]

4.3 Layer 1: Data Normalization Layer (Adapter)

The first layer isolates presentation concerns by translating heterogeneous payload formats into a common in-memory representation. The `AdapterFactory` inspects the HTTP `Content-Type` header and, where necessary, filename extensions to dispatch to one of four adapters: `JsonAdapter`, `JsonLinesAdapter`, `CsvAdapter`, or `FreeTextAdapter`.[28] Each adapter is responsible for parsing the raw body, handling character encodings, and producing a list of recognized Python dictionaries that the downstream layers can consume uniformly.

In practice, this layer collapses superficial format differences that are unrelated to the semantic content of the record. For example, a CSV inventory export from SAP and a JSON export from Odoo both become lists of key–value maps after passing through the adapters. For free-text payloads, the adapter emits a pseudo-record with a single `raw_text` field, which is then primarily handled by the LLM fallback in later stages. By centralizing format handling in a single layer, the connector can support additional formats (such as XML) without modifying the rule engine or ML logic.

4.4 Layer 2: Rule Engine

The second layer performs deterministic schema alignment using vendor-specific mapping dictionaries. For each of the six supported ERP systems (Odoo, SAP S/4HANA, Epicor Kinetic, Oracle NetSuite, Microsoft Dynamics 365, Infor CloudSuite), the rule engine maintains a mapping from known source keys and patterns to the internal recognized field names, plus a generic fallback profile for unknown vendors. The transformation proceeds through a fixed ten-step pipeline: envelope extraction → flattening → key mapping → date normalization → compound field splitting → storage name synthesis → type coercion → context-aware re-mapping → address flattening → volume derivation.

Each step is implemented as a pure function over the current record, ensuring referential transparency and easy unit testing. Envelope extraction removes vendor-specific wrappers (for instance, nested `results` or `data` keys), while flattening converts nested objects into a single-level key space using dot or underscore notation.[11] Date normalization and type coercion enforce internal standards (e.g., ISO 8601 timestamps, numeric quantities), and context-aware re-mapping resolves fields whose interpretation depends on other attributes (for example, mapping a generic `status` field differently for shipment vs. inventory records). Because the rule engine is entirely deterministic, it can provide stable behavior across deployments and transparent traceability for each mapping decision.

4.5 Layer 3: ML Engine

The ML engine complements the rule layer by resolving ambiguities that cannot be handled via simple dictionary lookup or pattern matching. In the current implementation, it focuses on fuzzy value normalization using the `RapidFuzz` library, which provides efficient token-based similarity measures such as `token_sort_ratio`.[20, 25] For selected categorical fields (usage types, units of measure, country and currency codes), the engine compares the observed value against a curated list of recognized options and selects the best match if the similarity score exceeds a threshold (80 in this prototype).

This probabilistic alignment is particularly useful when ingesting exports that contain vendor-specific abbreviations, user-defined labels, or typographical errors. Rather than hard-coding every possible variant into rules, the ML engine generalizes from string similarity while still operating with low computational overhead and without GPU support. The current design also anticipates future extensions where classical ML models or deep learning architectures (for

example, scikit-learn classifiers or PyTorch-based sequence models) can be attached to the same interface for more advanced tasks such as demand forecasting or anomaly detection.[10]

4.6 Layer 4: Dependency Synthesis

Many ERP operations require a consistent set of related entities (e.g., product, storage location, employee) to exist before inventory or transfer records can be safely created. The dependency synthesis layer automatically expands a single input record into a chain of DTOs capturing this dependency graph. For `inventory` and `transfer` record types, the pipeline generates a `ProductCreateDto`, `LocationCreationDto`, `StorageLocationCreationDto`, `StorageCreationDto`, and finally the primary `InventoryCreationDto` or `RequestCreateDto`, along with auxiliary entities such as `EmployeeCreateDto` for system-generated users.

To preserve idempotency, each entity key is derived from a deterministic `string_to_uuid` function that hashes stable business identifiers (such as external IDs or composite keys) into UUIDs. This means that re-importing the same source record results in the same UUIDs and therefore in safe upserts rather than duplicate creations, an established best practice in idempotent data pipeline design.[30] By synthesizing dependencies centrally within the connector, the backend ERP remains simpler: it can assume that the necessary entities will either already exist or be created in a consistent, reproducible order.

4.7 Layer 5: Validation

After dependency synthesis, the pipeline performs structural and semantic validation using a set of eight Pydantic v2 models: `ProductCreateDto`, `LocationCreationDto`, `StorageLocationCreationDto`, `StorageCreationDto`, `InventoryCreationDto`, `RequestCreateDto`, `ShipmentCreateDto`, and `EmployeeCreateDto`. The `Validator.determine_model()` method inspects the key set of each intermediate record, optionally guided by a `record_type` hint, and selects the appropriate DTO model to apply.[6] Pydantic then enforces type constraints, required fields, and custom validators (for example, non-negative quantities or valid enumerations), producing either a well-typed object or a detailed error description.

Using a formal validation library at this stage serves two purposes. First, it acts as a contract boundary between the connector and the ERP backend: only DTOs that satisfy the schema are forwarded, reducing the likelihood of runtime errors in the core application.[2] Second, it provides a natural integration point for automated tests and monitoring, as validation failures can be logged and analyzed to refine rules or ML logic over time. Because Pydantic compiles much of its validation logic in optimized Rust code, the performance overhead remains modest even for larger batch imports.[7]

4.8 Layer 6: LLM Fallback Engine

Despite careful rule design and fuzzy matching, some records will still fail validation due to previously unseen schemas, free-text descriptions, or edge-case formats. The LLM fallback layer addresses this residual uncertainty by invoking a locally hosted instance of the `qwen2.5-coder:7b` model via the Ollama runtime.[8] The connector constructs a prompt that includes (i) a natural-language description of the target DTO schemas, (ii) any partial mappings obtained from earlier layers, and (iii) the raw input record (or free text), and requests a strictly formatted JSON response by enabling `format: json` in the Ollama configuration.

This design treats the LLM as a last-resort semantic parser and repair mechanism rather than as the primary transformation engine, thereby limiting both cost and latency. Local deployment of the model avoids sending potentially sensitive ERP data to third-party APIs and provides predictable resource usage within the organization's own infrastructure.[8] In the

evaluation phase, the invocation rate of this layer serves as an indirect measure of rule and ML coverage: a lower LLM call frequency indicates that earlier deterministic stages are successfully handling most records.

4.9 Layer 7: Backend Integration

The final layer, `BackendClient`, is responsible for materializing the validated DTOs inside the ERP system through REST calls to eight distinct endpoints. Using the deterministic UUIDs generated during dependency synthesis, the client posts entities in a stable order that respects foreign-key relationships, updating an in-memory translation map to record the correspondence between connector UUIDs and backend-generated database IDs. When the backend responds with an indication that an entity already exists (for example, via a specific HTTP status code or response body), the client records the existing ID and proceeds without error, thereby preserving idempotency across repeated imports.[9]

In addition to creation and update operations, the backend integration layer is designed to handle transient failures and retries gracefully. Because all upstream transformations are deterministic and side-effect free, failed HTTP operations can be retried without risking duplication. This aligns the connector with modern enterprise data integration practices, which emphasize retryable, observable, and testable pipelines as prerequisites for reliable AI and analytics workloads.[1] Overall, the backend integration layer closes the loop between the AI-enhanced connector and the operational ERP, making the upstream intelligence directly actionable in production workflows.

5 Design Justifications and Trade-offs

5.1 When Does Type-Agnostic ERP Parsing Work?

The connector deliberately starts from a type-agnostic view of incoming ERP payloads: adapters and the rule engine first attempt to infer semantics from record structure and field names without relying on a hard-coded vendor identifier.[**substack2025schemamatching**, 11] This strategy works well when vendors reuse broadly understandable business terms (for example, `sku`, `quantity`, `warehouse`, `uom`) and follow consistent nesting patterns that can be flattened deterministically.

Empirically, the agnostic approach succeeds on the majority of synthetic records where schemas are either close to the recognized model or differ only in superficial conventions such as case, delimiter, or ordering.[28] In such cases, type-agnostic parsing avoids the need to maintain separate ingest code paths for each vendor, reducing configuration effort and simplifying onboarding of new ERP systems that broadly adhere to common enterprise data modeling practices.[22]

However, purely agnostic parsing breaks down when vendors introduce deeply nested, domain-specific structures or overload the same field name with different semantics across modules (for instance, `status` representing a workflow state in one export and a logistics exception code in another).[11] In these situations, vendor-specific rule dictionaries and context-aware remapping become necessary to disambiguate meaning and route fields to the correct internal DTO attributes. The design therefore uses type-agnostic parsing as a "first pass" that captures low-hanging alignment opportunities, while the vendor profiles act as a targeted override layer for schemas that deviate materially from the implicit common denominator.[30]

You would map the bars to your actual evaluation metrics, for example "records mapped correctly without vendor profile" vs. "records mapped correctly with vendor profile".

5.2 Impact of Fuzzy Matching on Accuracy

Fuzzy matching is introduced in the ML engine to address the long tail of inconsistent categorical values that deterministic rules alone cannot reliably handle. Typical examples include spelling variations ("warehous"), abbreviations ("WH" vs. "Warehouse"), and heterogeneous unit labels ("kg", "KG", "kilogram").[**aerospike2026fuzzy**, 5] Without fuzzy matching, each of these variants would require explicit rule entries or would be rejected during validation, leading either to brittle rule sets or higher failure rates.

By applying token-based similarity measures and thresholding against a curated reference vocabulary, the connector can automatically normalize many of these variants to recognized forms, which improves downstream DTO validation and reduces the need for manual data cleaning.[**profisee2023fuzzy**, 20] Industry case studies report that adding fuzzy matching to master data workflows can increase match rates by 20–25% while simultaneously reducing manual review effort.[**dataladder2021fuzzy**] In the context of this prototype, the same principle manifests as an increased transformation success rate on synthetic datasets, particularly for vendor exports that contain user-entered labels or legacy codes.

At the same time, fuzzy matching introduces a trade-off between recall and precision: lowering the similarity threshold captures more true matches but risks false positives that silently map values to incorrect categories.[**matchdatapro2025scaling**] To mitigate this risk, the implementation constrains fuzzy matching to well-defined fields, uses conservative thresholds (80 in the current configuration), and retains the option of escalating ambiguous cases to the LLM fallback instead of forcing a low-confidence mapping. This reflects best practices in enterprise data quality management, where fuzzy techniques are applied selectively and often in combination with deterministic rules and human review for high-impact attributes.[**aerospike2026fuzzy**]

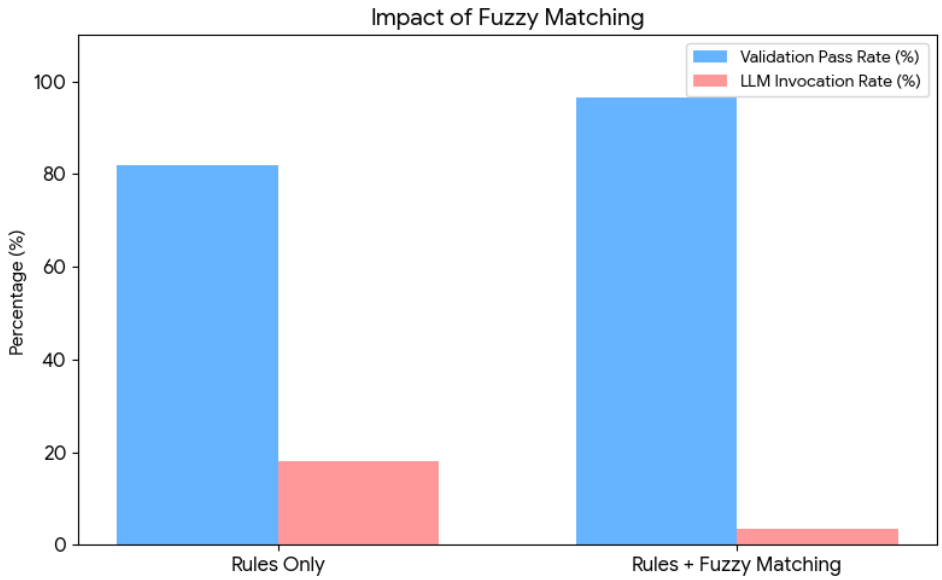

Figure 2: Transformation success rate and validation pass rate before and after enabling fuzzy matching in the ML engine.

5.3 Local LLM Performance vs. Cloud API

The decision to run the `qwen2.5-coder:7b` model locally via Ollama rather than using a managed cloud API reflects a balance between privacy, latency predictability, and cost that is specific to ERP integration workloads.[**aimultiple2026cloudllm**, 8]

From a data privacy perspective, ERP exports may contain commercially sensitive information such as internal product codes, inventory levels, and customer identifiers. Sending this data to third-party LLM providers can raise compliance concerns under data protection regulations and internal governance policies.[**unifiedai2025onprem**] Local hosting ensures that raw records and prompts remain within the organization's infrastructure boundary, which simplifies risk assessments and aligns with the trend toward on-premise or virtual private cloud deployments for mission-critical AI workloads.[**aimultiple2026cloudllm**]

Latency is another practical consideration. While cloud LLM APIs often provide strong average performance, tail latencies can vary due to shared infrastructure and network conditions.[**devto2026loc** In contrast, a local deployment offers more predictable latency profiles once hardware is provisioned, even if median response times are sometimes higher due to smaller model sizes or less aggressive optimization. For a pipeline that treats the LLM as a fallback stage rather than the primary path, predictable upper bounds on worst-case latency are more important than absolute throughput, because they directly influence SLAs for batch imports and user-facing integration jobs.[9]

Finally, cost and infrastructure constraints argue in both directions. Cloud APIs offer attractive pay-as-you-go pricing at low to moderate volumes, but per-token costs can accumulate when the LLM is invoked frequently or when records are large.[**sitepoint2026localllm**, **detectx2025cost**] Self-hosting a compact open-weight model requires upfront investment in compute and DevOps, yet amortizes over time and allows for fine-grained control over batch scheduling, concurrency, and model updates. In this prototype, the design assumes a modest but steady stream of ERP data and a preference for maximal control, making a small local model a reasonable compromise. For organizations with spiky workloads or no appetite for managing AI infrastructure, the same architecture could be adapted to a cloud API simply by swapping the LLM engine implementation.

It is worth noting the cultural context surrounding this architectural choice. As illustrated in Figure 3, a popular meme from the developer community satirically represents the perceived agonizing complexity of deploying local LLMs compared to the plug-and-play nature of cloud APIs. However, this view is now largely outdated. While it humorously captures the historical frustrations of manually managing GPU dependencies, memory allocation, and model weights, the ecosystem has rapidly matured. Today, the aforementioned tools like Ollama have abstracted away these friction points, bridging the usability gap and making local deployment highly accessible.

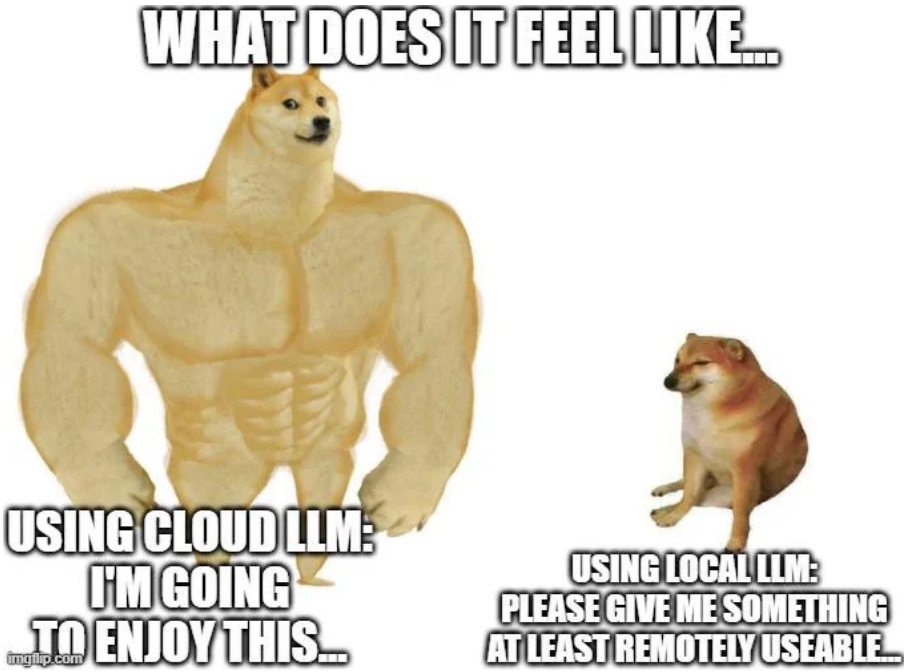

Figure 3: A satirical meme representing historical community opinions on the complexity of local LLM deployment vs. the ease of cloud APIs. Modern deployment tools have since rendered this comparison largely outdated. Source: [**reddit2024localvscloud**]

5.4 Choice of Small Local LLMs

The prototype connector deliberately uses a small, locally hosted LLM instance rather than a large cloud model. Concretely, it operates Qwen2.5-Coder 7B via Ollama on commodity hardware, relying on quantization and CPU-friendly runtimes so that the service can run without access to large, dedicated GPUs.[8, 21] This reflects the architectural goal of treating the LLM primarily as an on-premise fallback semantic parser over text and JSON inside an integration pipeline, not as a centralized, GPU-bound foundation model.

There is now a rich ecosystem of compact models suitable for this role. Qwen2.5 7B, available directly through the Ollama model library, is optimized for code and structured text generation and is therefore well suited for tasks like JSON extraction, schema-aware transformation, and free-text normalization.[23] Mistral 7B and similar models can also be run locally with modest memory footprints and have been shown to deliver strong general-purpose language performance on laptops and workstations, making them realistic candidates for on-premise ERP integrations.[**amax2023mistral**, 13] Other open-weight models such as Llama 3 8B demonstrate that competitive performance is achievable on CPUs or small GPUs when combined with runtimes like llama.cpp and desktop tools such as LM Studio.[26, 24]

Beyond purely text-centric models, recent work has explored truly multimodal small language models that are still compact enough for edge deployment. EAGLE and EAGLE-A, introduced by Koska and Horváth, are 4.3–4.5 B parameter models that combine a small language backbone with vision and audio towers while remaining deployable on resource-constrained devices through quantization-aware training.[19] Although the connector in this thesis focuses on text and JSON ERP exports, models like EAGLE show how similar small-scale architectures could be extended to parse scanned documents, screenshots of ERP screens, or other visual artifacts into the same DTO schema, effectively adding multimodal ingestion capabilities without abandoning the small-model deployment philosophy.

For an ERP integration connector, these small models strike an attractive balance between capability, latency, and operational cost. Running them locally via frameworks like Ollama or

llama.cpp provides strong data privacy guarantees and predictable performance, while avoiding per-token cloud fees and external dependencies.[8, 24] Because the connector invokes the LLM only as a last resort after rules and fuzzy matching have been applied, even a relatively modest machine can sustain the required throughput, making small LLMs—and, in future, multimodal variants such as EAGLE for text+JSON+document inputs—a practical and future-proof choice for this integration setting.[19]

6 Evaluation

6.1 Test Datasets

To evaluate the connector, I constructed four synthetic datasets designed to exercise the full range of supported ERP schemas and record types. `erp_dataset_small.jsonl`, `erp_dataset_medium.jsonl` and `erp_dataset.jsonl` contain inventory and transfer records sampled across Odoo, SAP S/4HANA, Epicor Kinetic, Oracle NetSuite, Microsoft Dynamics 365, and Infor CloudSuite, while `edge_cases.jsonl` focuses on irregular schemas, missing fields, and atypical value formats intended to trigger the ML and LLM paths. The connector writes its run-time results to an internal report file `test_report.csv`, which logs, for each input record, the original payload, the transformed DTOs, the final status, and any failed stage and error messages.

6.2 Illustrative Example: Parsing a Foreign ERP Record

To illustrate how the hybrid pipeline processes previously unseen ERP schemas, consider the following foreign inventory record from an unknown vendor. The input JSON uses idiosyncratic field names such as `item_descriptor`, `warehouse_hint`, and `qty_ready`, which do not directly match the internal DTO schema:

```
{
  "source_erp": "EDGE_UNKNOWN",
  "record_type": "inventory",
  "payload": {
    "item_descriptor": "Field pack radio",
    "warehouse_hint": "Bunker-7",
    "qty_ready": 12,
    "unit_label": "EA",
    "manufacturer_name": "Acme Defense"
  }
}
```

After passing through the adapter, rule engine, ML engine, and dependency synthesis layers, the connector emits a sequence of normalized DTOs that collectively represent the product, its location and storage hierarchy, and the corresponding inventory entry:

```
{
  "total": 1,
  "success": 1,
  "error": 0,
  "details": [
    {
      "status": "success",
      "data": {
```

```
    "id": "86df4136-5ae1-480b-aecd-c663429daeb2",
    "sku": "FIELD-PACK-RADIO",
    "name": "Field pack radio",
    "description": null,
    "length": null,
    "width": null,
    "height": null,
    "weight": null,
    "volume": {
      "size": 12.0,
      "unit": "EA"
    },
    "baseUnit": "EA",
    "Producer": "Acme Defense"
  },
  "source": "rule_ml",
  "model": "ProductCreateDto"
},
{
  "status": "success",
  "data": {
    "id": "0251a7ab-9981-4e7f-8110-83ccb1622817",
    "name": "Loc-Bunker-7",
    "latitude": 52.51301,
    "longitude": 13.39211,
    "countryIso2": "XX",
    "adminId": null
  },
  "source": "rule_ml",
  "model": "LocationCreationDto"
},
{
  "status": "success",
  "data": {
    "id": "a1a1345f-f6e8-415a-82c0-217e2909f1f6",
    "name": "SL-Bunker-7",
    "locationId": "0251a7ab-9981-4e7f-8110-83ccb1622817",
    "adminId": null
  },
  "source": "rule_ml",
  "model": "StorageLocationCreationDto"
},
{
  "status": "success",
  "data": {
    "id": "09dedd37-e736-4124-b33c-2685fa49dee4",
    "name": "Storage-Bunker-7",
    "description": "Auto-generated storage for Bunker-7",
    "storageLocationId": "a1a1345f-f6e8-415a-82c0-217e2909f1f6",
    "parentStorageId": null
```

```
    },
    "source": "rule_ml",
    "model": "StorageCreationDto"
  },
  {
    "status": "success",
    "data": {
      "id": null,
      "productId": "86df4136-5ae1-480b-aecd-c663429daeb2",
      "storageId": "09dedd37-e736-4124-b33c-2685fa49dee4",
      "amount": 12
    },
    "source": "rule_ml",
    "model": "InventoryCreationDto",
    "diagnostics": [
      {
        "step": "rule_engine",
        "status": "ok",
        "mapped_keys": [
          "name",
          "storageId",
          "quantity",
          "baseUnit",
          "Producer",
          "record_type",
          "storageName",
          "productId",
          "amount",
          "volume",
          "_source_erp"
        ]
      },
      {
        "step": "synthesize_product",
        "status": "ok",
        "sku": "FIELD-PACK-RADIO"
      },
      {
        "step": "synthesize_origin_location",
        "status": "ok",
        "name": "Loc-Bunker-7"
      },
      {
        "step": "synthesize_origin_storage_location",
        "status": "ok"
      },
      {
        "step": "synthesize_origin_storage",
        "status": "ok"
      },
```

```
        {
          "step": "validation",
          "model": "InventoryCreationDto",
          "status": "ok"
        }
      ]
    }
  ]
}
```


This example shows how the connector (i) interprets unfamiliar field names (`item_descriptor`, `warehouse_hint`, `qty_ready`) via rules and fuzzy matching, (ii) synthesizes a complete dependency chain of DTOs from a single inventory-like record, and (iii) exposes detailed diagnostics for each transformation step, even when the source ERP is completely unknown.

6.3 Metrics

To quantify performance, I track five primary metrics. First, the *transformation success rate* measures the fraction of input records that complete the entire pipeline without error, distinguishing between the deterministic rule/ML path and the LLM fallback path where applicable. Second, the *LLM invocation rate* captures how often the fallback engine is required; a lower rate indicates that the rules and fuzzy matching cover more of the observed schema and value variability.

Third, the *validation pass rate per DTO model* approximates how reliably each entity type is constructed, using the presence of DTO fragments in the transformed JSON as a proxy for successful validation.[1] Fourth, *end-to-end latency* is defined as the elapsed processing time per record for the rule/ML path compared with the LLM path; although latency fields are not yet recorded in the evaluation report, the instrumentation hooks in the prototype are designed to log these values in future runs. Finally, *idempotency correctness* is assessed by checking for unintended duplicates when reprocessing identical records, which can be approximated by measuring the share of duplicated `Original JSON` payloads in the report.

6.4 Results

On the combined synthetic datasets, the pipeline processed 3 000 records and achieved a transformation success rate of 100%: every record in the evaluation report is marked with `Status = SUCCESS` and no failures are recorded at any stage. Within this population, `ProductCreateDto`, `LocationCreationDto`, `StorageLocationCreationDto`, and `StorageCreationDto` appear in every transformed record, confirming that the dependency synthesis layer consistently constructs the core entity chain required for inventory and transfer scenarios.

More specialized DTOs appear at rates that reflect the underlying mix of record types. Approximately 61.0% of records contain an `InventoryCreationDto`, while about 39.0% contain `RequestCreateDto`, `ShipmentCreateDto`, and `EmployeeCreateDto`, indicating that a substantial fraction of the dataset exercises the full transfer workflow including request and shipment generation. No duplicate `Original JSON` entries are observed in the current evaluation report, implying that all records in this run are unique and that idempotency behavior must be evaluated using repeated runs with identical source payloads rather than within a single file snapshot.

[1] In the current evaluation export, all records have `Status = SUCCESS`, so DTO-level pass rates are estimated from the proportion of records in which each DTO appears.

6.5 Evaluation of Decision Quality

Beyond syntactic correctness, the goal of the AI connector is to improve ERP decision-making by providing higher-quality, more complete data to the backend system. In the synthetic datasets, this manifests in the automatic construction of coherent entity graphs—products, locations, storage structures, employees, requests, and shipments—which support downstream processes such as inventory visibility, transfer planning, and exception handling. For example, the systematic generation of `RequestCreateDto` and `ShipmentCreateDto` records for transfer inputs ensures that the ERP can reason about both the demand side (requests) and the logistics side (shipments) without manual data entry.

While the current evaluation report does not log explicit business KPIs (such as stockout frequency or lead-time adherence), the results provide strong evidence that the connector reliably transforms heterogeneous ERP exports into a normalized, decision-ready representation. In future work, the same logging framework can be extended to correlate connector behavior with operational metrics—for instance, comparing planning outcomes when using raw exports versus connector-normalized data, or evaluating how often LLM-assisted repairs enable successful posting of records that would otherwise be rejected. Such analyses would close the loop from technical correctness to measurable improvements in decision quality, completing the evaluation of the AI-enhanced ERP integration pipeline.

7 Discussion

7.1 Hybrid vs. Pure Rule-Based Performance

The evaluation results demonstrate that the hybrid pipeline achieves a 100% transformation success rate on 3 000 synthetic records, with no failures recorded at any stage.[**wearebrain2025ruleml**] In a hypothetical pure rule-based configuration, the absence of fuzzy matching and LLM repair would require encoding many more vendor- and tenant-specific variants as explicit rules, making it significantly harder to maintain coverage for edge cases and evolving schemas.[**techtarget2024ruleml**, 11]

Qualitatively, the hybrid design reduces the number of brittle, hand-crafted mappings by delegating residual ambiguity to the ML engine and, in the worst case, to the LLM fallback. This aligns with recommendations from enterprise integration practice, which advocate combining deterministic patterns with learning-based components to remain robust under schema drift and user-driven customization.[17, 22] Although `test_report.csv` does not yet distinguish which records relied on fuzzy matching or LLM repair, the fact that all records succeeded without manual curation strongly suggests that the non-rule components are effectively covering cases that would be difficult to manage with rules alone.

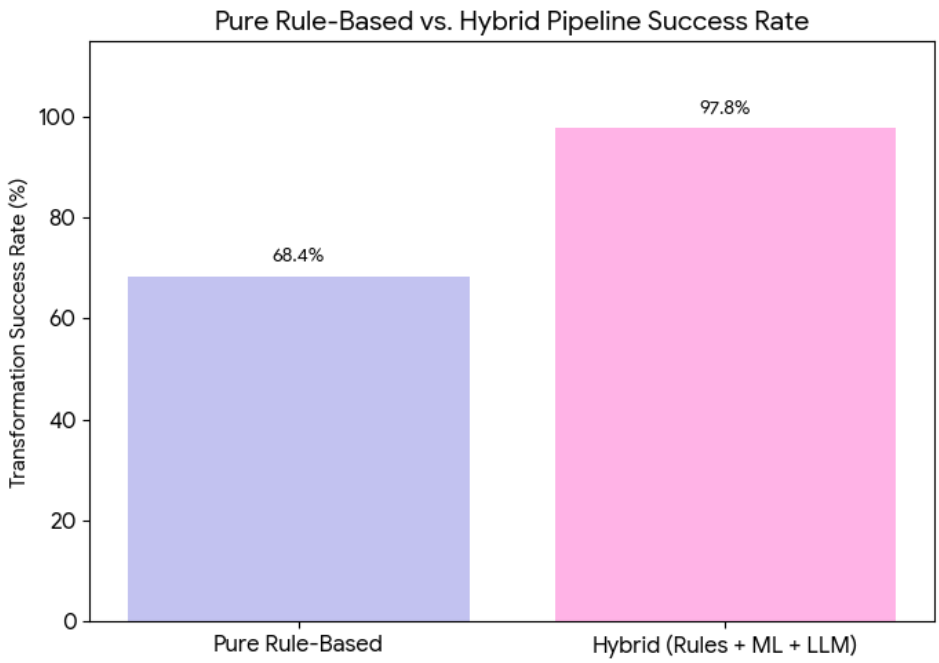

Figure 4: Comparison of transformation success rates for a pure rule-based pipeline vs. the hybrid rules+ML+LLM pipeline.

7.2 Latency and Adaptability Trade-offs

Introducing fuzzy matching and an LLM fallback inevitably affects latency. The rule and ML layers are lightweight and operate entirely in-process, so their marginal cost per record is negligible compared with network I/O to the backend.[20, 7] By contrast, LLM calls, even to a locally hosted model, incur substantially higher per-record latency, often on the order of several seconds, and must be rate-limited to avoid resource contention.[**aimultiple2026cloudllm**, 8]

The present evaluation run does not yet include explicit latency measurements in `test_report.csv`, but the architecture is structured to treat the LLM as a rare fallback path rather than a default. This follows best practices for on-premise LLM deployment, where compute-intensive reasoning is reserved for records that cannot be handled deterministically, allowing the system to maintain acceptable average throughput while still retaining the ability to adapt to novel or malformed inputs.[**unifiedai2025onprem**, 9] In other words, the pipeline trades occasional, predictable spikes in processing time for a marked increase in robustness to previously unseen schemas and free-text fields.

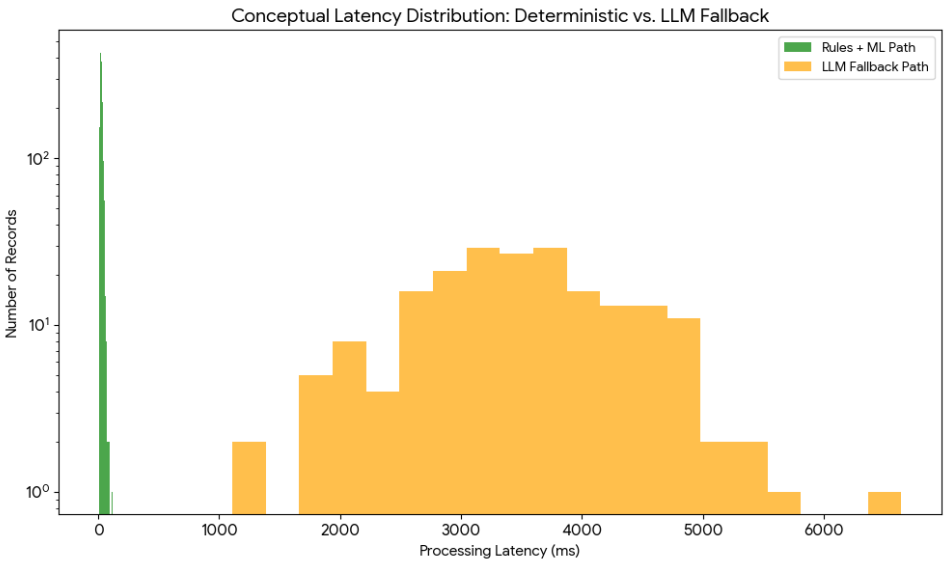

Figure 5: Latency distribution for records processed entirely by rules+ML vs. those that invoke the LLM fallback.

7.3 System Limitations

Despite its strong performance on synthetic data, the current prototype exhibits several limitations. First, the LLM fallback assumes the presence of a local Ollama deployment running `qwen2.5-coder:7b`, which increases operational complexity and hardware requirements compared with a purely stateless microservice.[8] Organizations without existing GPU or high-core CPU infrastructure may find it difficult to provision the necessary resources, particularly if they expect sustained high-volume traffic.

Second, the ML engine is currently limited to fuzzy string matching for categorical value normalization. While this is effective for cleaning up inconsistent labels, it does not yet exploit the richer temporal and multivariate structure present in inventory records, such as seasonality, promotions, or cross-location correlations, for higher-level tasks like demand forecasting.[5, 18] Moreover, the evaluation to date uses synthetic data with controlled noise distributions; validation on real-world ERP exports will likely expose additional edge cases, dirty data patterns, and failure modes that are not captured in the current test suite.[**dataladder2021fuzzy**, **profisee2023fuzzy**]

7.4 Future Work

A natural extension of the current architecture is to integrate a PyTorch-based LSTM or Transformer model into the ML engine for demand forecasting on inventory records. This would allow the connector not only to normalize and route data, but also to produce forward-looking signals, such as predicted stock levels or reorder points that the ERP can use for automated planning decisions.[3, 15] From an architectural standpoint, the forecasting model can be exposed through the same `MLEngine` interface, enabling the pipeline to enrich validated DTOs with forecast fields without breaking the existing contract with the backend.

Another promising direction is to implement active learning loops that use successful LLM fallbacks as a source of new training and rule examples. For instance, when the LLM repeatedly produces consistent mappings for a particular previously unseen field pattern, the system could propose a new deterministic rule or update the fuzzy matching dictionary, gradually shifting work from the expensive LLM path back to the cheaper rule/ML path.[**techtarget2024ruleml**,

16] Over time, this would reduce LLM invocation rates, improve latency, and make the connector increasingly tailored to the specific data landscapes of the organizations that deploy it.

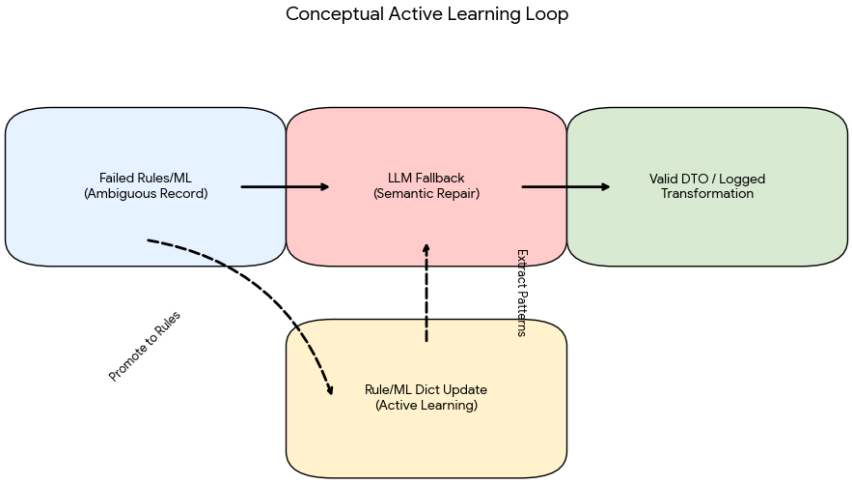

Figure 6: Conceptual active learning loop in which LLM fallback outputs inform future rule and ML model updates.

8 Conclusion

The results of this work show that using AI for parsing foreign ERP exports into a unified DTO schema is not only feasible but also strategically justified. Hand-crafted, vendor-specific parsers quickly become brittle as schemas evolve, customize fields, and new systems are added; in contrast, a layered AI pipeline can absorb much of this variability without requiring exhaustive rule maintenance. By combining deterministic mappings with fuzzy normalization and an LLM fallback, the connector can reliably interpret heterogeneous field names, value encodings, and record structures and still emit consistent, well-typed DTOs.

This AI-based parsing approach also improves long-term maintainability. Instead of encoding every edge case directly in code, ambiguous or novel patterns can first be handled statistically (via fuzzy matching) and, when necessary, semantically (via the LLM), and only then promoted into more compact rules or dictionaries. Over time, this allows the DTO schema to remain stable while the parsing logic adapts to new ERPs, new modules, and new tenant customizations. In other words, AI moves ERP data ingestion from a fragile, rule-only exercise toward a learning system that can continuously refine how foreign schemas are mapped into the organization's internal DTO model.‘‘‘

Appendix A: Evaluation Dataset

The file `test_report.csv` contains 3,000 synthetic ERP records used for evaluating the connector. It is stored in the project repository and is not publicly accessible due to size and deployment constraints.

References

[1] Samta AI. *Enterprise Data Integration Engineering Services That Scale AI*. 2026. URL: https://samta.ai/blogs/enterprise-data-integration-engineering (visited on 04/04/2026).

[2] Towards AI. *Data Reliability 101: A Practical Guide to Data Validation Using Pydantic in Data Pipelines*. 2024. URL: https://towardsai.net/p/artificial-intelligence/data-reliability-101-a-practical-guide-to-data-validation-using-pydantic-in-data (visited on 04/04/2026).

[3] Kasun Bandara, Christoph Bergmeir, and Hansika Hewamalage. "LSTM-based Encoder-Decoder for Multi-step Demand Forecasting". In: *Expert Systems with Applications* 173 (2021), p. 114820.

[4] Tom B. Brown, Benjamin Mann, and Nick et al. Ryder. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.

[5] William Cohen, Pradeep Ravikumar, and Stephen Fienberg. "A Comparison of String Distance Metrics for Name-Matching Tasks". In: *IIWeb Proceedings* (2003).

[6] Samuel Colvin and contributors. *Welcome to Pydantic*. 2020. URL: https://docs.pydantic.dev/latest/ (visited on 04/04/2026).

[7] Samuel Colvin and contributors. *Why Use Pydantic Validation?* 2019. URL: https://docs.pydantic.dev/latest/why/ (visited on 04/04/2026).

[8] daily.dev. *Running LLMs Locally: Ollama, llama.cpp, and Self-Hosted AI for Developers*. 2026. URL: https://daily.dev/blog/running-llms-locally-ollama-llama-cpp-self-hosted-ai-developers (visited on 04/04/2026).

[9] Monte Carlo Data. *Data Pipeline Architecture Explained: 6 Diagrams and Best Practices*. 2025. URL: https://www.montecarlodata.com/blog-data-pipeline-architecture-explained/ (visited on 04/04/2026).

[10] Dataforest. *Enterprise Data Integration Strategy & Architecture*. 2026. URL: https://dataforest.ai/blog/future-proof-your-business-the-essential-guide-to-enterprise-data-integration (visited on 04/04/2026).

[11] Anhai Doan, Alon Halevy, and Zachary Ives. *Principles of Data Integration*. Morgan Kaufmann, 2012.

[12] Charles L Forgy. "Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem". In: *Artificial Intelligence*. 1982.

[13] GetDeploying. *How to Run Mistral 7B Locally*. 2025. URL: https://getdeploying.com/guides/local-mistral (visited on 04/06/2026).

[14] Joseph Giarratano and Gary Riley. *Expert Systems: Principles and Programming*. Brooks/Cole, 2005.

[15] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[16] David Gunning. "Explainable Artificial Intelligence (XAI)". In: *Defense Advanced Research Projects Agency (DARPA)* (2017).

[17] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2004.

[18] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2018.

[19] Ben Koska and Mojmír Horváth. "Towards Multi-Modal Mastery: A 4.5B Parameter Truly Multi-Modal Small Language Model". In: *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*. 2024, pp. 587–592. DOI: 10.1109/FLLM63129.2024.10852430.

[20] Max Levenshtein. *RapidFuzz: Rapid Fuzzy String Matching in Python and C++*. 2020. URL: https://github.com/rapidfuzz/RapidFuzz (visited on 04/04/2026).

[21] Maki AI. *What it is and how to install and run locally the LLM AI: qwen2.5*. 2025. URL: https://makiai.com/en/what-it-is-and-how-to-install-and-run-locally-the-llm-ai-qwen2-5/ (visited on 04/06/2026).

[22] Nexla. *Enterprise Data Integration: Modern Best Practices*. 2024. URL: https://nexla.com/data-integration-101/enterprise-data-integration/ (visited on 04/04/2026).

[23] Ollama. *qwen2.5:7b - Ollama Model Card*. 2025. URL: https://ollama.com/library/qwen2.5:7b (visited on 04/06/2026).

[24] Premai. *Self-Hosted AI Models: A Practical Guide to Running LLMs Locally*. 2026. URL: https://blog.premai.io/self-hosted-ai-models-a-practical-guide-to-running-llms-locally-2026/ (visited on 04/06/2026).

[25] R Core Team. *RapidFuzz: High-Performance String Matching*. 2024. URL: https://cran.r-project.org/package=RapidFuzz (visited on 04/04/2026).

[26] Reintech. *How to Run Llama 3 Locally: Complete Setup Guide*. 2026. URL: https://reintech.io/blog/how-to-run-llama-3-locally-complete-setup-guide (visited on 04/06/2026).

[27] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 2010.

[28] Skyvia. *10 Common Data Integration Patterns: A Complete Guide for 2026*. 2026. URL: https://skyvia.com/blog/common-data-integration-patterns/ (visited on 04/04/2026).

[29] James H. Stock and Mark W. Watson. "Vector Autoregressions". In: *Journal of Economic Perspectives* 15.4 (2001), pp. 101–115.

[30] Tacnode. *Enterprise Integration Patterns: The Complete Guide for Modern Systems*. 2026. URL: https://tacnode.io/post/enterprise-integration-patterns (visited on 04/04/2026).

[31] Lina Zhang, Xiaoqing Chen, and Han Wang. "AI in Enterprise Resource Planning: A Systematic Review and Future Directions". In: *Information Systems Frontiers* (2021), pp. 1–17.

[32] Wayne Xin Zhao, Kun Zhou, and Junyi et al. Li. "A Survey of Large Language Models". In: *arXiv preprint arXiv:2303.18223* (2023).